

Démarrage — Lancer Open TutorAI en local (sans Docker)

Ce guide accompagne un débutant complet, étape par étape, pour faire tourner Open TutorAI sur son propre ordinateur. Suivez chaque étape dans l'ordre et l'application sera opérationnelle.

Ce que vous obtiendrez à la fin

Quoi	Adresse
Interface web (Frontend)	http://localhost:5173
API backend	http://localhost:8080
Documentation interactive de l'API	http://localhost:8080/docs

Étape 0 — Vérifier les prérequis

Vous avez besoin de deux logiciels installés avant de commencer.

Python 3.11 ou 3.12

Pourquoi pas la dernière version de Python ?

Certaines bibliothèques utilisées par ce projet n'ont pas encore de paquets précompilés pour Python 3.13 ou 3.14. Vous devez utiliser **3.11 ou 3.12**.

Vérifiez votre version :

```
python --version
```

- Si la réponse est `Python 3.11.x` ou `Python 3.12.x` → vous êtes prêt.

- Si la réponse est `Python 3.13.x` ou `Python 3.14.x` → installez Python 3.11 depuis <https://www.python.org/downloads/release/python-3119/>
- Si la commande n'est pas reconnue → installez Python 3.11 depuis le lien ci-dessus.

Windows uniquement : Après avoir installé plusieurs versions de Python, vous pouvez choisir laquelle utiliser avec :

```
py --list
```

Cela affiche toutes les versions installées. Vous utiliserez `py -3.11` (et non `python`) dans les commandes ci-dessous.

Node.js 18 ou supérieur

Vérifiez :

```
node --version
```

- Si la réponse est `v18.x.x` ou supérieure → vous êtes prêt.
- Sinon → installez Node.js depuis <https://nodejs.org> (choisissez la version « LTS »).

Étape 1 — Récupérer le code source

Si vous n'avez pas encore cloné le dépôt :

```
git clone https://github.com/Open-TutorAi/open-tutor-ai-CE.git
cd open-tutor-ai-CE
```

Toutes les commandes suivantes supposent que vous êtes **dans le dossier** `open-tutor-ai-CE`.

Étape 2 — Créer le fichier de configuration

L'application lit ses paramètres dans un fichier appelé `.env`.

Copiez le fichier d'exemple pour le créer :

Mac / Linux :

```
cp .env.example .env
```

Windows (PowerShell) :

```
Copy-Item .env.example .env
```

Les paramètres par défaut fonctionnent très bien en développement local.
Vous n'avez rien à modifier pour l'instant.

Étape 3 — Créer un environnement virtuel Python

Un environnement virtuel isole les paquets Python de ce projet du reste de votre ordinateur.

Mac / Linux :

```
python3.11 -m venv .venv
```

Windows — si `python --version` affiche déjà 3.11 ou 3.12 :

```
python -m venv .venv
```

Windows — si vous avez plusieurs versions Python (utilisez le lanceur py) :

```
py -3.11 -m venv .venv
```

À faire une seule fois. Un dossier `.venv` apparaîtra dans le projet.

Étape 4 — Installer les dépendances Python

Utilisez directement l'exécutable pip de l'environnement virtuel.

Mac / Linux :

```
.venv/bin/pip install -r requirements-ci.txt
.venv/bin/pip install requests==2.32.3 aiohttp==3.11.11 async-timeout aiocache \
    alembic==1.14.0 argon2-cffi==23.1.0 APScheduler==3.10.4 \
    RestrictedPython==8.0 openai anthropic tiktoken
```

Windows (PowerShell) :

```
.venv\Scripts\pip.exe install -r requirements-ci.txt
.venv\Scripts\pip.exe install requests==2.32.3 aiohttp==3.11.11 async-timeout aiocache `
    alembic==1.14.0 argon2-cffi==23.1.0 APScheduler==3.10.4 `
    RestrictedPython==8.0 openai anthropic tiktoken
```

Pourquoi deux commandes ?

`requirements-ci.txt` contient les paquets de base. La deuxième commande ajoute les paquets nécessaires pour faire tourner l'application (clients IA, planificateur de tâches, etc.). On ignore `psycopg2` (le pilote PostgreSQL) car le développement local utilise SQLite — pas besoin de PostgreSQL.

Cette étape prend 2 à 5 minutes. Beaucoup de texte s'affiche — c'est normal.

Étape 5 — Créer les dossiers de données

L'application stocke les fichiers téléversés et sa base de données dans un dossier `var/`. Créez-le :

Mac / Linux :

```
mkdir -p var/uploads var/vector_db
```

Windows (PowerShell) :

```
New-Item -ItemType Directory -Force var\uploads
New-Item -ItemType Directory -Force var\vector_db
```

Étape 6 — Installer les dépendances du frontend

```
cd ui
npm install
cd ..
```

Cette étape prend 1 à 3 minutes et installe environ 1 000 paquets.

Des avertissements de sécurité peuvent apparaître — c'est normal, ils n'affectent pas le développement local.

Étape 7 — Lancer l'application

Vous avez besoin de **deux fenêtres de terminal ouvertes en même temps**, toutes deux pointant vers le dossier du projet.

Terminal 1 — Backend (API Python)

Mac / Linux :

```
.venv/bin/uvicorn main:app --reload --port 8080
```

Windows (PowerShell) :

```
.venv\Scripts\uvicorn.exe main:app --reload --port 8080
```

Attendez d'avoir la ligne :

```
INFO:      Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

Terminal 2 — Frontend (Interface SvelteKit)

```
cd ui
```

```
npm run dev
```

Attendez d'avoir la ligne :

```
→ Local: http://localhost:5173/
```

Étape 8 — Ouvrir l'application et créer votre compte

Ouvrez votre navigateur et allez sur : **<http://localhost:5173>**

Une page d'inscription apparaît. Créez un compte —

le premier compte créé devient automatiquement administrateur.

Étape 9 — Ajouter des modèles d'IA

L'application a besoin d'un modèle d'IA pour fonctionner.

Vous avez deux options :

Option A — Ollama (gratuit, tourne sur votre ordinateur)

Idéal pour : utilisation hors ligne, confidentialité, aucun coût d'API.

Nécessite : un ordinateur récent (minimum 8 Go de RAM).

1. Téléchargez et installez Ollama depuis <https://ollama.com/download>
2. Après l'installation, ouvrez un nouveau terminal et téléchargez un modèle :

```
bash ollama pull llama3.2
```

Cela télécharge environ 2 Go. D'autres bons modèles : `mistral` , `qwen2.5:3b` .

1. Ollama tourne automatiquement en arrière-plan sur `http://localhost:11434` .
2. Dans l'application, allez dans **Admin** → **Paramètres** → **Fournisseurs**.
Vous verrez `http://localhost:11434` déjà renseigné. Cliquez sur **Enregistrer**.

3. Retournez dans le chat — votre modèle apparaît dans le menu déroulant de sélection.

Option B — API Cloud (OpenAI, GroqCloud, etc.)

Idéal pour : des modèles puissants sans ordinateur haut de gamme.

Nécessite : une clé API auprès du fournisseur.

1. Dans l'application, allez dans **Admin** → **Paramètres** → **Fournisseurs**.
2. Activez **OpenAI**.
3. Renseignez l'URL et la clé API de votre fournisseur :

Fournisseur	URL de base	Où obtenir une clé
OpenAI	<code>https://api.openai.com/v1</code>	<code>https://platform.openai.com</code>
GroqCloud (offre gratuite)	<code>https://api.groq.com/openai/v1</code>	<code>https://console.groq.com</code>
LMStudio (local)	<code>http://localhost:1234/v1</code>	Aucune clé requise

1. Cliquez sur **Enregistrer**. Les modèles de ce fournisseur apparaissent immédiatement dans le sélecteur du chat.

Comment relancer l'application les fois suivantes

Les étapes 1 à 6 ne sont à faire qu'une seule fois. Pour relancer, ouvrez deux terminaux et exécutez :

Terminal 1 (backend) :

```
# Mac / Linux
.venv/bin/uvicorn main:app --reload --port 8080

# Windows
.venv\Scripts\uvicorn.exe main:app --reload --port 8080
```

Terminal 2 (frontend) :

```
cd ui  
npm run dev
```

Résolution de problèmes

« Module not found » ou erreurs d'import au démarrage du backend

Assurez-vous de lancer uvicorn depuis le dossier racine du projet (pas depuis `ui/` ni un sous-dossier), et d'utiliser `.venv/Scripts/uvicorn.exe` (pas une installation globale).

La liste des modèles est vide

- **Ollama** : Vérifiez qu'Ollama est en cours d'exécution (`ollama list` dans un terminal doit afficher vos modèles).
- **OpenAI** : Vérifiez que vous avez enregistré la clé API dans Admin → Paramètres → Fournisseurs et que la clé est valide.

Port déjà utilisé

Si vous voyez `address already in use` pour le port 8080 ou 5173, un autre processus utilise ce port. Arrêtez-le, ou changez de port :

```
.venv/bin/uvicorn main:app --reload --port 8081
```

Puis ajoutez `http://localhost:5173` dans `CORS_ALLOW_ORIGIN` dans votre fichier `.env`.

Le frontend affiche « Impossible de se connecter à l'API »

Le backend doit être démarré avant d'utiliser le frontend.
Vérifiez que le Terminal 1 est toujours actif et ne présente pas d'erreurs.

Windows : `py -3.11` introuvable

Installez Python 3.11 depuis <https://www.python.org/downloads/release/python-3119/> et cochez « Add Python to PATH » pendant l'installation. Puis vérifiez avec `py --list`.

Récapitulatif — Rôle de chaque composant

Composant	Rôle	Port
Backend	Serveur Python FastAPI — gère les données, l'auth et les appels IA	8080
Frontend	Application web SvelteKit — l'interface visible dans le navigateur	5173
Ollama	Serveur de modèles IA local (optionnel)	11434
Base SQLite	Créée automatiquement dans <code>var/tutorai.db</code> au premier lancement	—